



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2233 Programación Avanzada (2026-2)

Ejercicios propuestos 1

Programación orientada a objetos

■ Entrega

- **Fecha y hora límite:** 4 de septiembre de 2026, hasta las 20:00 horas.
- **Lugar:** Repositorio personal de GitHub. El código debe encontrarse en la rama (*branch*) por defecto del repositorio: `main`.
- **Contacto:** Para informar que los entregables se encuentran disponibles y solicitar feedback, debes comunicarte con el/la profesor/a de tu respectiva sección. No se aceptarán entregables enviados después de las **20:00 horas del 4 de septiembre de 2026**.
- **Condiciones de revisión:** Solo se entregará feedback a las entregas que cumplan con las condiciones de ejecución, formato, legibilidad y organización indicadas en la [sección de Restricciones y alcances](#).

Objetivos

- Aplicar conceptos de programación orientada a objetos (POO) para modelar y resolver un problema.
- Distribuir datos y responsabilidades entre los distintos componentes del programa.
- Representar elementos que responden de manera diferente ante una misma acción.
- Utilizar estructuras de datos adecuadas para resolver los distintos requerimientos.
- Procesar *input* del usuario de forma robusta, manejando posibles errores de formato.
- Trabajar con archivos para leer, modificar y conservar el estado del programa.
- Familiarizarse con el proceso de entrega, el uso de GIT y las buenas prácticas de programación.

Índice

1. Introducción	3
2. El centro de rescate	3
2.1. Recintos	3
2.2. Animales	4
3. Consola	4
3.1. Consultar el estado	4
4. Iniciar una jornada	5
4.1. Solicitud de ingreso	5
4.2. Acciones del centro	6
4.3. Cierre de la jornada	6
5. Liberación de animales	7
6. Archivos del programa	7
6.1. <code>centro.csv</code>	7
6.2. <code>recintos.csv</code>	7
6.3. <code>animales.csv</code>	8
6.4. Guardar cambios	8
7. Parámetros	8
8. Restricciones y alcances	8

1. Introducción

En distintos lugares de Chile existen centros dedicados a recibir animales silvestres que han sufrido accidentes, enfermedades o la pérdida de su hábitat. Estos centros deben decidir qué animales pueden recibir, cómo distribuir sus recursos y qué casos necesitan atención prioritaria.

En este ejercicio propuesto desarrollarás *DCCRescate*, un programa de consola para administrar un centro de rescate. Durante cada jornada llegará una solicitud de ingreso y tendrás que decidir qué hacer con ella. Luego podrás utilizar una cantidad limitada de acciones para alimentar animales, realizar tratamientos o mantener los recintos en buenas condiciones.

El enunciado describe las reglas que debe cumplir el programa, pero no establece una cantidad específica de clases ni una jerarquía predeterminada. La elección de cómo representar y relacionar los elementos del sistema será parte del ejercicio.

2. El centro de rescate

El programa administra un único centro de rescate. El centro posee:

- Un nombre.
- Una cantidad de créditos disponibles.
- Una reputación entre 0 y 100.
- El número de la jornada actual.
- Una colección de recintos.
- Una cantidad acumulada de animales liberados.

Al iniciar el programa, el nombre, los créditos, la reputación, la jornada y la cantidad de animales liberados deben obtenerse desde `centro.csv`. Los valores de este archivo representan el estado actual del centro y no deben ser reemplazados por valores definidos en `parametros.py`.

Los créditos permiten alimentar animales, realizar tratamientos, limpiar recintos, derivar solicitudes y construir instalaciones. La reputación representa la confianza que la comunidad tiene en el centro y cambia como consecuencia de las decisiones tomadas.

Todo intento de modificar la reputación debe mantenerla dentro del intervalo cerrado $[0, 100]$.

2.1. Recintos

Cada recinto posee un nombre único, un ambiente, una capacidad máxima, un nivel de limpieza entre 0 y 100, los animales que aloja y, opcionalmente, una instalación.

Existen dos ambientes:

- **Terrestre:** permite alojar mamíferos.
- **Aviario:** permite alojar aves.

Un recinto nunca puede superar su capacidad. Además, su limpieza debe permanecer dentro del intervalo $[0, 100]$ después de cualquier operación.

Cada recinto puede tener como máximo una de las siguientes instalaciones:

- **Clínica:** cuesta `COSTO_CLINICA` créditos y mejora los tratamientos realizados en ese recinto.

- **Bodega:** cuesta `COSTO_BODEGA` créditos y reduce en un 20 % el costo de alimentar a sus animales. El resultado se aproxima al entero inferior.

2.2. Animales

Todo animal posee un identificador único, un nombre, una edad, un nivel de salud, un nivel de hambre y el recinto donde se encuentra. Tanto la salud como el hambre son números enteros entre 0 y 100. Un número mayor de hambre representa una condición más desfavorable.

Inicialmente el centro puede recibir mamíferos y aves. Sus características se muestran en la siguiente tabla:

Tipo	Ambiente	Alimentación	Tratamiento	Bono clínica
Mamífero	Terrestre	30 créditos	25 de salud	10 de salud
Ave	Aviario	20 créditos	15 de salud	20 de salud

Las reglas particulares de cada tipo deben estar concentradas en componentes apropiados del programa. No debes repetir comparaciones sobre el tipo del animal en todos los menús y operaciones. Incorporar un nuevo tipo de animal debería requerir cambios localizados.

La forma concreta de lograr lo anterior queda a criterio de quien implemente la solución.

3. Consola

El programa se ejecuta mediante:

```
python3 main.py
```

Al iniciarse debe mostrar un menú similar al siguiente:

```
=== DCCRescate ===
Jornada actual: 4
Créditos: 420 | Reputación: 63

[1] Ver estado del centro
[2] Ver recintos y animales
[3] Iniciar jornada
[4] Liberar un animal
[5] Guardar cambios
[0] Salir
```

Puedes modificar los textos y la presentación, pero todas las opciones indicadas deben estar disponibles.

3.1. Consultar el estado

La primera opción muestra los créditos, la reputación, la jornada actual, la cantidad de animales alojados, la cantidad de animales liberados y el promedio de salud de los animales actuales, redondeado a un decimal.

La segunda opción muestra, para cada recinto, su ambiente, limpieza, instalación, ocupación y los animales que contiene. Para cada animal se debe indicar su identificador, nombre, tipo, salud y hambre.

Cuando no existan animales o recintos, se debe mostrar un mensaje apropiado.

4. Iniciar una jornada

Esta es la opción principal del programa. Una jornada se divide en tres etapas que siempre se ejecutan en el orden indicado.

4.1. Solicitud de ingreso

Al comenzar la jornada se genera un animal que necesita ayuda. Su información se crea de la siguiente manera:

- El tipo se escoge aleatoriamente entre mamífero y ave, con igual probabilidad.
- El nombre se escoge aleatoriamente desde una lista definida en `parametros.py`.
- La edad se genera entre 0 y `EDAD_MAXIMA_SOLICITUD`.
- La salud se genera entre `SALUD_MINIMA_SOLICITUD` y `SALUD_MAXIMA_SOLICITUD`.
- El hambre se genera entre `HAMBRE_MINIMA_SOLICITUD` y `HAMBRE_MAXIMA_SOLICITUD`.

El programa debe mostrar estos datos antes de solicitar una decisión:

```
--- Nueva solicitud ---
```

```
Nombre: Chucao
```

```
Tipo: Ave
```

```
Edad: 2
```

```
Salud: 38
```

```
Hambre: 71
```

```
[1] Admitir en el centro
```

```
[2] Derivar a otro centro
```

```
[3] Rechazar la solicitud
```

Cada decisión tiene consecuencias:

- **Admitir:** cuesta `COSTO_ADMISION` créditos y aumenta la reputación en `REPUTACION_POR_ADMISION` puntos. El usuario debe escoger un recinto compatible con espacio disponible. Si no existe uno, esta alternativa no puede realizarse.
- **Derivar:** cuesta `COSTO_DERIVACION` créditos.
La reputación aumenta en `REPUTACION_POR_DERIVACION` punto y el animal no ingresa al centro. Esta alternativa no puede realizarse si no hay créditos suficientes.
- **Rechazar:** no cuesta créditos, pero disminuye la reputación en `PERDIDA_POR_RECHAZO` puntos. El animal no ingresa al centro.

Si el usuario selecciona una alternativa que no puede realizarse, debe volver a escoger. La solicitud siempre debe terminar con una de las tres decisiones aplicadas.

4.2. Acciones del centro

Luego de resolver la solicitud, el usuario dispone de `ACCIONES_POR_JORNADA` acciones. En cada una puede escoger una de las siguientes alternativas:

```
--- Acción 1 de 2 ---  
[1] Alimentar un animal  
[2] Tratar un animal  
[3] Limpiar un recinto  
[4] Construir una instalación  
[5] No realizar una acción
```

Las alternativas funcionan de la siguiente forma:

- **Alimentar:** descuenta el costo correspondiente al tipo de animal y reduce su hambre en `REDUCCION_HAMBRE` puntos. Si el recinto posee una bodega, se aplica su descuento.
- **Tratar:** cuesta `COSTO_TRATAMIENTO` créditos. El animal recupera la salud correspondiente a su tipo. Si se encuentra en un recinto con clínica, se suma el bono indicado anteriormente.
- **Limpiar:** cuesta `COSTO_LIMPIEZA` créditos y aumenta la limpieza del recinto en `AUMENTO_LIMPIEZA` puntos.
- **Construir:** permite escoger una clínica o una bodega para un recinto que todavía no tenga instalación. Se descuenta su costo.
- **No realizar una acción:** consume la acción sin modificar el centro.

Una acción inválida no consume el turno. Por ejemplo, intentar alimentar sin créditos suficientes debe mostrar un mensaje y permitir escoger nuevamente.

El límite de acciones obliga al usuario a decidir qué necesidad atender primero. Es válido realizar dos veces la misma clase de acción si las condiciones lo permiten.

4.3. Cierre de la jornada

Cuando se terminan las acciones se aplican, en orden, las consecuencias diarias:

1. El hambre de todos los animales aumenta en `AUMENTO_HAMBRE_DIARIO` puntos.
2. La limpieza de todos los recintos disminuye en `REDUCCION_LIMPIEZA_DIARIA` puntos.
3. Todo animal cuyo nivel de hambre sea 100 pierde `DANO_POR_HAMBRE` puntos de salud.
4. Los animales de un recinto cuya limpieza sea 0 pierden `DANO_POR_SUCIEDAD` puntos de salud.

Si un animal recibe ambos efectos, pierde salud por ambas causas. Si su salud llega a 0, debe ser derivado automáticamente a un centro especializado: se elimina del recinto y la reputación disminuye en `PERDIDA_POR_DERIVACION_URGENTE` puntos.

Finalmente se incrementa el número de la jornada y se imprime un resumen de la solicitud, las acciones ejecutadas y los cambios de salud, hambre, limpieza, créditos y reputación.

5. Liberación de animales

Desde el menú principal se puede intentar liberar uno de los animales alojados. Para hacerlo, debe cumplir simultáneamente:

- Tener una salud mayor o igual a `SALUD_MINIMA_LIBERACION`.
- Tener un hambre menor o igual a `HAMBRE_MAXIMA_LIBERACION`.

Al liberar un animal, se elimina del recinto, aumenta la cantidad acumulada de animales liberados, se reciben `RECOMPENSA_LIBERACION` créditos y la reputación aumenta en `REPUTACION_POR_LIBERACION` puntos.

Si el animal no cumple las condiciones, el programa debe mostrar qué requisitos faltan sin modificar el estado.

6. Archivos del programa

Al iniciar, el programa carga su estado desde tres archivos CSV. Puedes utilizar el módulo `csv` de Python y asumir que los archivos entregados existen, poseen el orden de columnas indicado y contienen datos válidos.

Para todos los archivos se cumplen las siguientes condiciones:

- Están codificados en UTF-8 y utilizan una coma (,) como separador.
- La primera línea corresponde al encabezado y no representa un elemento del programa.
- Los valores simbólicos, como `terrestre`, `aviario`, `mamifero`, `ave`, `clinica`, `bodega` y `ninguna`, se escriben en minúsculas, sin tildes y exactamente como aparecen en esta sección.
- Los nombres pueden contener espacios. No se agregan espacios adicionales antes o después de las comas separadoras.

6.1. `centro.csv`

Contiene un encabezado y una única fila de información del centro:

```
nombre,creditos,reputacion,jornada,liberados
Refugio Cordillera,1000,50,1,2
```

6.2. `recintos.csv`

Cada fila representa un recinto:

```
nombre,ambiente,capacidad,limpieza,instalacion
Bosque Norte,terrestre,4,80,ninguna
Pradera Sur,terrestre,2,55,bodega
Aviario Central,aviario,5,60,clinica
Humedal Alto,aviario,3,100,ninguna
```

El valor de `instalacion` será `clinica`, `bodega` o `ninguna`.

El valor de `ambiente` será `terrestre` o `aviario`. El nombre de cada recinto es único. Es válido que el archivo contenga solamente el encabezado cuando no existan recintos; en ese caso, `animales.csv` tampoco puede contener animales.

6.3. animales.csv

Cada fila representa un animal alojado:

```
id,nombre,tipo,edad,salud,hambre,recinto
1,Luna,mamifero,3,45,60,Bosque Norte
2,Pudu,mamifero,5,90,15,Pradera Sur
3,Zorro,mamifero,2,70,85,Bosque Norte
4,Quique,mamifero,4,82,20,Pradera Sur
5,Pepa,ave,2,70,30,Aviario Central
6,Condor,ave,8,35,95,Aviario Central
```

El valor de `tipo` será `mamifero` o `ave`. La columna `recinto` debe coincidir exactamente con el nombre de un recinto existente y compatible en `recintos.csv`.

Los identificadores son números enteros positivos y no se repiten. El identificador de una nueva admisión debe corresponder al mayor identificador existente más uno. Si no existen animales, el primer identificador será 1. Es válido que el archivo contenga solamente el encabezado cuando el centro no aloje animales.

6.4. Guardar cambios

La opción `Guardar cambios` sobrescribe los tres archivos con el estado actual. Debe conservar los mismos encabezados, el orden de columnas y los valores simbólicos indicados anteriormente. Los archivos escritos deben poder cargarse nuevamente en una ejecución posterior.

Salir sin guardar descarta los cambios realizados desde el último guardado. No es necesario implementar una confirmación adicional al salir.

7. Parámetros

Todos los valores mostrados en [ESTE_FORMATO](#), las listas utilizadas para generar solicitudes, los nombres de archivos y cualquier otro valor constante deben definirse en `parametros.py`.

Los siguientes valores permiten comenzar a probar el programa:

Parámetro	Valor	Parámetro	Valor
COSTO_ADMISION	60	COSTO_DERIVACION	25
REPUTACION_POR_ADMISION	2	REPUTACION_POR_DERIVACION	1
PERDIDA_POR_RECHAZO	2	ACCIONES_POR_JORNADA	2
COSTO_TRATAMIENTO	50	REDUCCION_HAMBRE	40
COSTO_LIMPIEZA	40	AUMENTO_LIMPIEZA	30
COSTO_CLINICA	300	COSTO_BODEGA	200
AUMENTO_HAMBRE_DIARIO	15	REDUCCION_LIMPIEZA_DIARIA	10
DANO_POR_HAMBRE	10	DANO_POR_SUCIEDAD	5
SALUD_MINIMA_LIBERACION	80	HAMBRE_MAXIMA_LIBERACION	20
RECOMPENSA_LIBERACION	150	REPUTACION_POR_LIBERACION	5

Importante: Queda a tu criterio los valores de los parámetros.

8. Restricciones y alcances

- Este ejercicio es **estrictamente individual** y está regido por el [Código de Honor](#) de Ingeniería.

- Tu programa debe ser desarrollado utilizando Python **3.12.X**, con X mayor o igual a 0.
- Tu programa debe estar compuesto por uno o más archivos con extensión `.py`, organizados coherentemente dentro de la carpeta de entrega. **No se revisarán archivos en otros formatos**, como `.ipynb`.
- El programa será ejecutado **únicamente desde la terminal**, ubicando la terminal dentro de la carpeta de entrega y utilizando el siguiente comando:

`python3 main.py`

- Durante la revisión se podrá cambiar el nombre de la carpeta que contiene el **Ejercicio Propuesto** por otro nombre. Por lo tanto, los *paths* relativos utilizados en el programa deben ser coherentes con esta instrucción.
- Solo se revisarán entregables que puedan ejecutarse correctamente desde la consola. Si el programa no puede iniciarse, se cierra inesperadamente antes de acceder a sus principales funcionalidades o requiere pasos no documentados para ejecutarse, el entregable podrá no ser revisado y no se entregará feedback.
- La consola debe ser robusta frente a opciones inválidas, texto en campos numéricos e identificadores inexistentes. Se permite utilizar `try/except` para validar conversiones y operaciones de archivos.
- Se espera que el código sea legible, esté organizado coherentemente y cumpla con las buenas prácticas de programación trabajadas en el curso. En particular, se espera:
 - Utilizar nombres descriptivos para variables, funciones, clases y módulos.
 - Distribuir las responsabilidades del programa de manera coherente.
 - Evitar la repetición innecesaria de código.
 - Evitar concentrar toda la solución en un único archivo, función o componente.
 - Mantener un formato consistente que facilite la lectura del código.
 - Documentar las decisiones que no sean evidentes.

Si el código no presenta condiciones mínimas de legibilidad u organización que permitan comprender su funcionamiento, el entregable podrá no ser revisado y no se entregará feedback.

- Debe existir una separación clara entre la interacción por consola, las reglas del centro y el manejo de archivos. Puedes distribuir el código en los módulos que consideres apropiados; no existe una estructura única obligatoria.
- Si no se encuentra especificado en el enunciado, supón que el uso de cualquier librería Python **está prohibido**. Pregunta en una *discussion* en el [foro](#) si es que es posible utilizar alguna librería en particular.
- Este ejercicio debe desarrollarse **exclusivamente con los contenidos liberados** al momento de publicar el enunciado. No se permite utilizar contenidos del curso liberados después de su publicación.
- Está prohibido citar o utilizar código publicado **después de la liberación del enunciado**. Solo se permite citar contenido que ya se encontrara disponible antes de su publicación.
- Ante cualquier duda relacionada con el enunciado, puedes utilizar [Discussions](#).
- Cualquier aspecto no especificado queda a tu criterio, siempre que no contradiga una regla explícita del enunciado. Los supuestos realizados deben aplicarse consistentemente y quedar explicados en el archivo `README.md`.

Los entregables que no cumplan con las restricciones de ejecución, formato o legibilidad podrán no ser revisados y no recibir feedback.